Workload Identity Federation Deck

https://docs.google.com/presentation/d/1pOZNrm8VVd943DdZtER2WHS3-zJRrtdVK2Gmk4IS1fA/edit?usp=sharing
and local copy:
https://docs.google.com/presentation/d/15Wh_aAQY97Bxat8MDavtmTdta4eeGRb76AI9p-TpT-o/edit?usp=sharing

Cloud Platform Support: k8s SA

https://g3doc.corp.google.com/company/gfw/support/cloud/jumpstarts/container-engine/k8s-service-account.md?cl=head

KSA: Kubernetes service accounts are part of the GKE cluster in which they are defined and it is a namespace scoped object

IAM Service Accounts: IAM service accounts are part of a Google Cloud project, and can be granted permissions both within clusters and to Google Cloud project, as well as to any Google Cloud resource, using Identity and Access Management (IAM).

When you create a namespace in the GKE cluster, it automatically creates a Kubernetes service account object named `default` in this namespace.
If you delete the `default` ServiceAccount object in a namespace, the control plane replaces it with a new one.

This `default` service account in each namespace gets no permissions by default other than the default API discovery permissions that Kubernetes grants to all authenticated principals if role-based access control (RBAC) is enabled.

Application Pods and system components can use a specific service account's credentials to identify as that service account.

You can create a service account using the following command.

```
kubectl create serviceaccount
```

When you create a service account in a GKE cluster, a service account token also gets generated. This token is what will be required by the application to access the Kubernetes APIs. You can view the token using the following command.

```
kubectl describe serviceaccount
```

```
raprasanth@cloudshell:~ (gkeproject-349106)$ kubectl describe serviceaccount myserviceaccount
Name:                myserviceaccount
Namespace:           default
Labels:              <none>
Annotations:         <none>
Image pull secrets:  <none>
Mountable secrets:   myserviceaccount-token-g8z2d
Tokens:              myserviceaccount-token-g8z2d
Events:              <none>
```

From the above output, you will see that the Tokens attribute is created with the value: `myserviceaccount-token-g8z2d`. This token is stored as a secret object, and this secret object is attached to the service account: `myserviceaccount`.

From version 1.22 onwards, Kubernetes introduced TokenRequest API. A token generated through this API is a time-bound token that expires after a time (by default, that is 1 hour). It applies to both the default service account and the custom-defined service accounts.
To view the secret object, you can execute the following command. You can view the encoded hash-key value of the token as highlighted below:

```
kubectl describe secret
```

```
raprasanth@cloudshell:~ (gkeproject-349106)$ kubectl describe secret myserviceaccount-token-g8z2d
Name:         myserviceaccount-token-g8z2d
Namespace:    default
Labels:       <none>
Annotations:  kubernetes.io/service-account.name: myserviceaccount
              kubernetes.io/service-account.uid: deac9170-f385-4541-b140-c70bc649eb8d

Type:  kubernetes.io/service-account-token

Data
====
ca.crt:     1509 bytes
namespace:  7 bytes
token:      eyJhbGciOiJSUzI1NiIsImtpZCI6InFEVVFlWGVkVGljbDV2UFBoMHNFNVZmYkJEV0lDb0lHOFZxaWhWWWIzajgifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2Nv
dW50IiwiaXZiZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYWlc3BhY2UiOiJkZWZhdWx0IiwiaXZiZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZWNyZXQubmFtZSI6Im15c2Vydml
jZWFjY291bnQtdG9rZW4tZzh6MmQiLCJrdWJlcm5ldGVzLmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC5uYW1lIjoibXlzZXJ2aWNlYWNjb3VudCIsImtlYmVybmV0ZXMuaW
8vc2Vydmljb2WFjY291bnQvc2Vydmljb2SlhY2NvdW50LnVpZCI6ImRlYWM5MTc4LWYzODUtNDU0MS1iMTQwLWM3MGJjNjQ5ZWI4ZCIsInNlYiI6InN5c3RlbTpzZXJ2aWNlYWNjb3VudDpkZ
WZhdWx0Om15c2Vydmljb2WFjY291bnQifQ.2RD0MUuCWA0_Vzilq8fruqpRkeDZzkBoM3XCX-bo4MNThNt6TkZOb9cqEu2nX1R3wv7uy-pOGhdCrbTQhrUPRh2wWLeCWceaRhGBySeSnUs3r
y67FGS2b2gGdd6AauJ-JfaRBvYJQ4DFNPE0oB0rPkhiXCfLTH5FgFwh_kiNnfh9k-TdLJBIjsMoi-QL8drYENXeQclaYZS1ei36aXwsRZJM4MjKgZJNhayNDCFme0oA_wJx2Ug0YV7Fykx6
-DbHOh9HAtajNbkXH_ZL3cQsWUSIV7RqkKUQDAVkdsjqOO027MKpKf_aKQ-Tgi4iBXe__-d7hHTPPBD0jbcEkEaoWQ
```

This is the key that can be exchanged as an authentication bearer token to access the Kubernetes API.

Now you grant permissions to the ServiceAccount object using an authorization mechanism such as RBAC.

Permissions are granted to a Service Account through Roles and RoleBindings (or ClusterRoles and ClusterRoleBindings for cluster-wide permissions):

- **Role:** Defines permissions within a namespace.
- **RoleBinding:** Binds a Role to a Service Account within a namespace.
- **ClusterRole:** Similar to Role but for cluster-wide permissions.

- **ClusterRoleBinding:** Binds a ClusterRole to a Service Account for cluster-wide permissions.

Here's an example of a Role and RoleBinding that grants a Service Account read-only access to Pods within a namespace:

Role:

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: my-namespace
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

Role binding:

```
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-pods
  namespace: my-namespace
subjects:
- kind: ServiceAccount
  name: my-service-account
  namespace: my-namespace
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Then, assign the ServiceAccount object to pods. You can find the example pod YAML file below.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - image: nginx
```

```
    name: nginx
  serviceAccountName: myserviceaccount
```

where the service account is:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-service-account
  namespace: my-namespace
```

and use the service account on the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
spec:
  serviceAccountName: my-service-account
  containers:
  - name: my-container
    image: my-image
```

k8s SA in action: Image pull error

https://rudimartinsen.com/2024/01/13/cka-notes-kubernetes-service-accounts-2024/

Cloud platform support: GKE WI

https://g3doc.corp.google.com/company/gfw/support/cloud/playbooks/container-engine/workload-identity.md?cl=head

Workflow details

go/gke-workload-identity-support#workflow-details

System pods and kubelet

Important

Important: Image pulls from Google Container Registry (GCR) or Artifact Registry (AR) by the kubelet (a process on the VM) do not use Workload Identity. Ensure that the node pool has the right access scopes configured and the service account used has the correct permissions to pull the image. The same applies to system pods like `fluentbit-gke` and `gke-metrics-agent` that use the node service account to access Cloud Logging and Monitoring respectively.

Important

System workloads and workloads with `hostNetwork: true` are configured to bypass MDS and instead route requests to the underlying node's metadata server.

Playbook (lots of deep details)

https://g3doc.corp.google.com/company/gfw/support/cloud/playbooks/container-engine/workload-identity.md?cl=head

GKE Training: WI and IAM

https://docs.google.com/presentation/d/1eQYoHKJJ_7V86DxDnNEFAc0O-9OAdFIqvOsVcfflPag/edit

https://docs.google.com/presentation/d/1_3Q7tw_lQLIB3e-vFXXTdgQTOMHksSeaUJu5I4fxIi8/edit?usp=sharing

Deck: Workload Identity 2.0

How it used to work

How it used to work

Google

Confidential + Proprietary Google

How it used to Work

1. Create a Kubernetes **namespace** and a **Kubernetes Service Account (KSA)** (optional).
2. Create a **Google Service Account (GSA)**.
3. Assign the necessary **IAM roles** to the **GSA**.
4. Add an **IAM policy binding** between the **GSA** and the **KSA**.
   a. KSA can impersonate the GSA.
5. **Annotate** the KSA with the GSA.

Google

Confidential + Proprietary

- Create a Kubernetes namespace and a Kubernetes Service Account (KSA) (optional).
- Create a Google Service Account (GSA).
- Assign the necessary IAM roles to the GSA.
- Add an IAM policy binding between the GSA and the KSA.
- KSA can impersonate the GSA.
- Annotate the KSA with the GSA.

Add an IAM policy binding between the GSA and the KSA

Google Service Account (GSA)

```
gcloud iam service-accounts add-iam-policy-binding workload-identity-gsa@gcp-support-project-1.iam.gserviceaccount.com \
    --role roles/iam.workloadIdentityUser \
    --member "serviceAccount:gcp-support-project-1.svc.id.goog[wi-namespace/wi-ksa]"
```

Workload Identity User Role

Workload Identity Pool

Kubernetes Namespace

Kubernetes Service Account

Confidential + Proprietary

What Changed

What Changed?

Google

Confidential + Proprietary Google

What Changed?

1. Create a Kubernetes **namespace** and a **Kubernetes Service Account (KSA)** (optional).
2. ~~Create a **Google Service Account (GSA)**~~
3. Assign the necessary **IAM roles** directly to the **KSA**
4. ~~Add an **IAM policy binding** between the **GSA** and the **KSA**~~
5. ~~**Annotate** the KSA with the GSA~~

Google

Confidential + Proprietary

Create a Kubernetes namespace and a Kubernetes Service Account (KSA) (optional).

- Create a Google Service Account (GSA)
- Assign the necessary IAM roles directly to the KSA

New KSA Principals

The new KSA principals

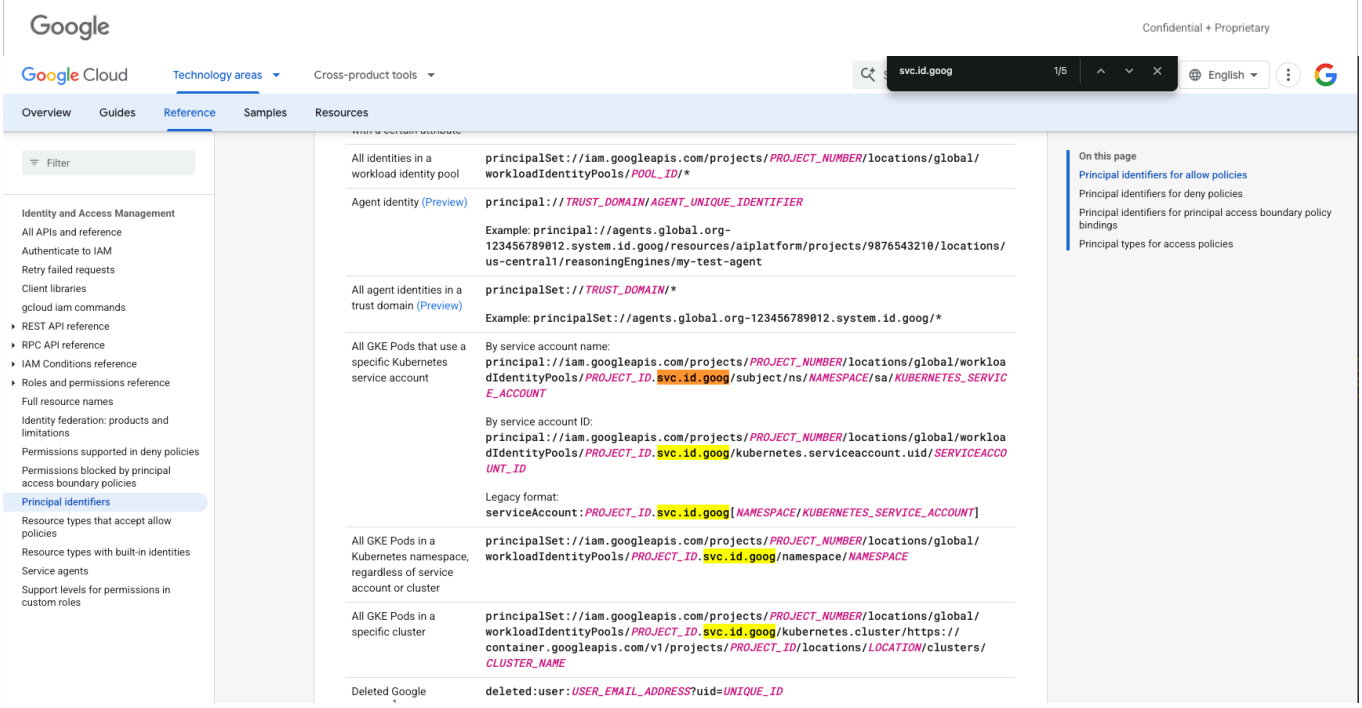
All Pods that use a specific Kubernetes ServiceAccount
principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/subject/ns/NAMESPACE/sa/SERVICEACCOUNT

All workloads in a cluster (regardless of KNS or KSA)
principalSet://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/kubernetes.cluster/https://container.googleapis.com/v1/projects/PROJECT_ID/locations/LOCATION/clusters/CLUSTER_NAME

All workloads in a namespace (regardless of KSA or cluster):
principalSet://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT-ID.svc.id.goog/namespace/KNS

One specific Kubernetes ServiceAccount by its UID:
principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/kubernetes.serviceaccount.uid/SERVICEACCOUNT_UID

And more..



and more
https://cloud.google.com/iam/docs/principal-identifiers

Identity and Access Management
All APIs and reference
Authenticate to IAM
Retry failed requests
Client libraries
gcloud iam commands
REST API reference
RPC API reference
IAM Conditions reference
Roles and permissions reference
Full resource names
Identity federation: products and limitations
Permissions supported in deny policies
Permissions blocked by principal access boundary policies
Principal identifiers
Resource types that accept allow policies
Resource types with built-in identities
Service agents
Support levels for permissions in custom roles

Principal identifiers

Send feedback

When you refer to a principal in an Identity and Access Management (IAM) policy, you need to use the correct identifier for the principal. The format of the identifier depends on the type of principal that you want to refer to and the type of policy that you're writing.

This page lists the identifier formats for each policy type's supported principal types.

Principal identifiers for allow policies

The following table describes the principal identifiers for allow policies, which use the IAM `v1` API.

These identifiers are also used for Privileged Access Manager entitlements.

Preview — Principal identifiers for all service accounts in a project, folder, or organization

This feature is subject to the "Pre-GA Offerings Terms" in the General Service Terms section of the Service Specific Terms. Pre-GA features are available "as is" and might have limited support. For more information, see the launch stage descriptions.

Principal type	Identifier
Google Account	`user:`*`USER_EMAIL_ADDRESS`* Example: `user:alex@example.com`
Service account	`serviceAccount:`*`SA_EMAIL_ADDRESS`* Example: `serviceAccount:my-service-account@my-project.iam.gserviceaccount.com`

All service accounts in a project, folder, or organization	`principalSet://cloudresourcemanager.googleapis.com/`*`RESOURCE_TYPE`*`/`*`RESOURCE_NUMBER`*`/type/ServiceAccount`
	Example for all service accounts in a project:
	`principalSet://cloudresourcemanager.googleapis.com/projects/123456789012/type/ServiceAccount`
	Example for all service accounts in all projects in a folder:
	`principalSet://cloudresourcemanager.googleapis.com/folders/123456789012/type/ServiceAccount`
	Example for all service accounts in all projects in an organization:
	`principalSet://cloudresourcemanager.googleapis.com/organizations/123456789012/type/ServiceAccount`
	Note: Moving projects into or out of a folder or organization changes the service accounts included in this principal set. For example, if you move a project out of a folder or organization, then this principal set no longer includes that project's service accounts.

Example

Example

1. Create a Kubernetes **namespace** and a **Kubernetes Service Account (KSA)** (optional).

```
kubectl create namespace test_ns
kubectl create serviceaccount test_ksa --namespace test_ns
```

2. Assign the necessary **IAM roles** directly to the **KSA**

```
gcloud projects add-iam-policy-binding projects/test_project \
      --role=roles/appengine.appAdmin \
      --member=principal://iam.googleapis.com/projects/255719237156/locations
      /global/workloadIdentityPools/test_project.svc.id.goog/subject/ns/test_
      ns/sa/test_ksa \
      --condition=None
```

Google

Confidential + Proprietary

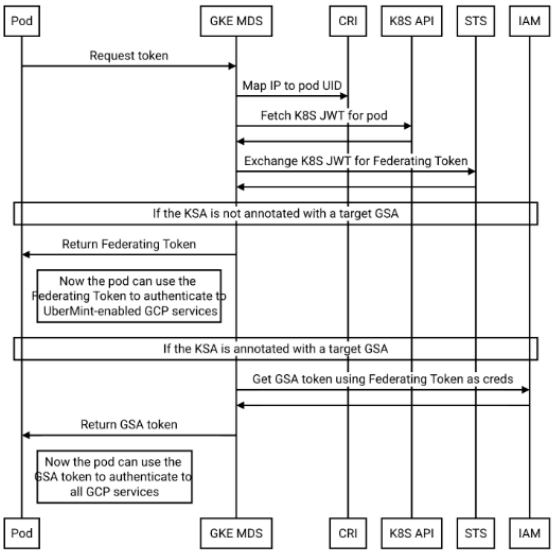
How is an Acccess Token Fetched

How is an access token fetched?

1. Workload calls:
   a. http://metadata.google.internal/..../service-accounts/default/token
2. iptables rule redirects the request to the GKE MDS pod listening on 127.0.0.1:989
3. GKE MDS identifies the source pod of the incoming request (source IP address), performs a look-up of the KSA that the pod is running as and observes an annotation declaring intent to run as a GSA.
4. GKE MDS makes a TokenRequest to the Kubernetes API for the KSA the pod is running as. The Kubernetes API returns a KSA token.
5. GKE MDS exchanges the KSA token for a Google federated access token using the GCP Secure Token Service API.

[If the KSA of the pod is annotated(*iam.gke.io/gcp-service-account*) with a GSA]

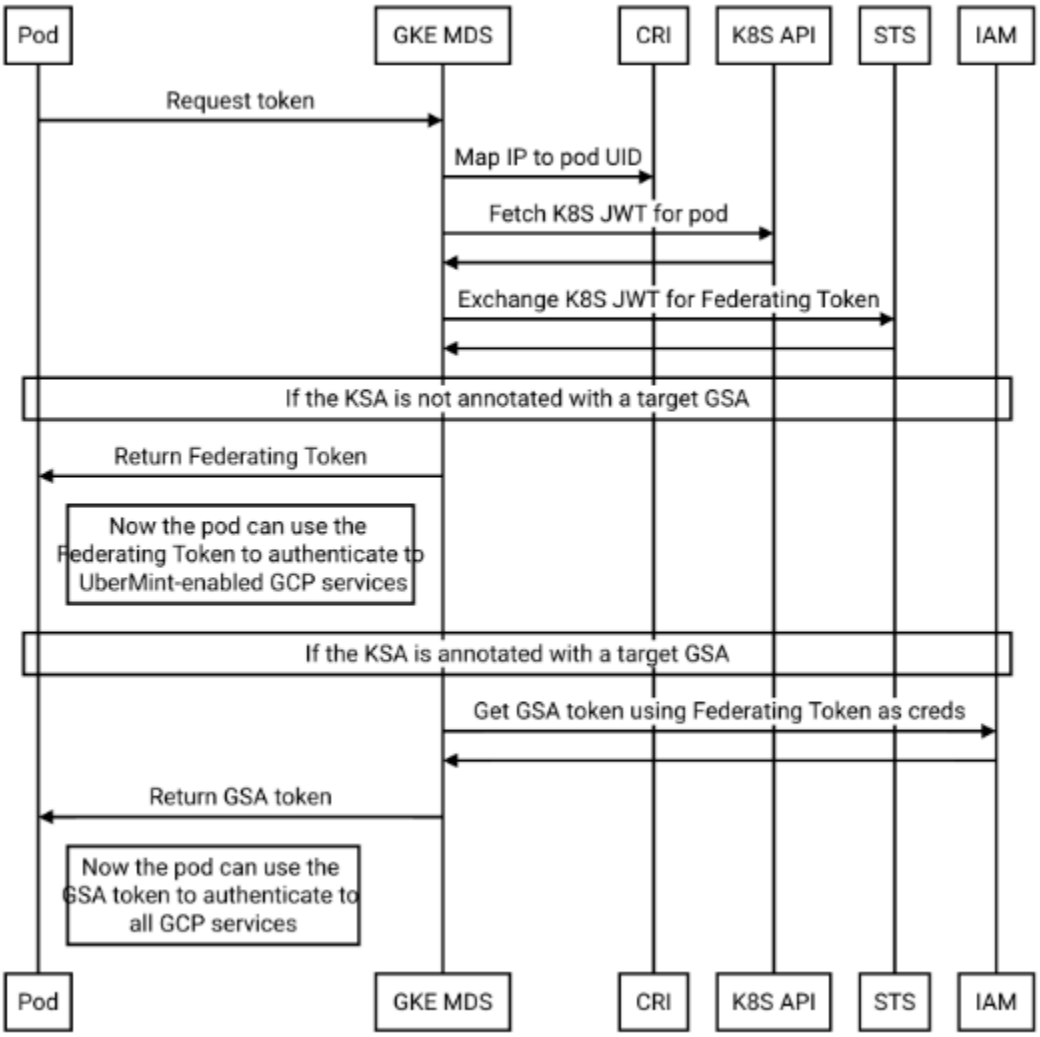
6. GKE MDS exchanges the federated access token for a Google access token for the KSA principal
7. GKE MDS returns Google access token/Federating token to Workload.
8. Workload uses the token for calls to GCP APIs.



Google

1. Workload calls: http://metadata.google.internal/…./service-accounts/default/token
2. iptables rule redirects the request to the GKE MDS pod listening on 127.0.0.1:989
3. GKE MDS (Metadata Server) identifies the source pod of the incoming request (source IP address), performs a look-up of the KSA that the pod is running as and observes an annotation declaring intent to run as a GSA.

4. GKE MDS makes a TokenRequest to the Kubernetes API for the KSA the pod is running as.
   The Kubernetes API returns a KSA token.
5. GKE MDS exchanges the KSA token for a Google federated access token using the GCP Secure Token Service API.
   [If the KSA of the pod is annotated(iam.gke.io/gcp-service-account) with a GSA]
6. GKE MDS exchanges the federated access token for a Google access token for the KSA principal
   GKE MDS returns Google access token/Federating token to Workload.
7. Workload uses the token for calls to GCP APIs.



Do I need to update my existing resources?

No...

- All existing legacy KSA+GSA bindings would remain working and new ones can be created even now.
- You could have a mix of legacy method and updated method in the same cluster (identity pool)

Google

Confidential + Proprietary

Limitations

Limitation

- Not all GCP Products support Identity federation, ref: *Identity federation: products and limitations*
- Use the legacy methods if the use case falls into using an unsupported product. Ref: *Alternative: link Kubernetes ServiceAccounts to IAM*

Google

Confidential + Proprietary

Alternative./ Legacy Method

https://docs.cloud.google.com/kubernetes-engine/docs/how-to/workload-identity#kubernetes-sa-to-iam

Solution for APIs that don't support k8s Tokens

- Create a Google Service Account (GSA)
- Add KSA to the IAM role iam.workloadIdentityUser* allowing it to obtain tokens for the GSA
  - permission: 'iam.serviceAccounts.getAccessToken'
  - permission: 'iam.serviceAccounts.getCertificateAs'
  - permission: 'iam.serviceAccounts.getOpenIdToken'
- Transparently exchange KSA tokens for GSA tokens
- Workload can now access GCP APIs by presenting GSA tokens

```
* gcloud iam service-accounts add-iam-policy-binding \
  --role roles/iam.workloadIdentityUser \
  --member "serviceAccount:cluster_project.svc.id.goog[k8s_namespace/ksa_name]" \
  gsa_name@gsa_project.iam.gserviceaccount.com
```

Federation Identity Supported services.

When a service is not supported by Federation Identity, then one need to use the legacy method

https://docs.cloud.google.com/iam/docs/federated-identity-supported-services#cloud-shell

Based on the "Identity federation: products and limitations" documentation, the following Google Cloud services are explicitly listed as **Unsupported** for identity federation:

- **App Engine**: Google recommends using Cloud Run as an alternative (source).
- **Cloud Code**: There are no recommended alternatives provided.
- **Cloud Deployment Manager**: No alternatives are currently available.
- **Cloud Endpoints**: No alternatives are currently available.
- **Cloud Mobile App**: No alternatives are currently available (source).
- **Cloud Profiler**: No alternatives are currently available.
- **Cloud Shell**: Google recommends using Cloud Workstations as an alternative (source).
- **Cloud Source Repositories**: No alternatives are listed in this document.
- **Endpoint Verification**: No alternatives are currently available.
- **Enterprise Knowledge Graph**: No alternatives are currently available.
- **Google Distributed Cloud**: No alternatives are currently available.
- **Looker (Google Cloud core)**: No alternatives are currently available (source).
- **Looker Studio**: No alternatives are currently available.
- **Translation Hub**: No alternatives are currently available.

Principal Types

https://docs.cloud.google.com/iam/docs/principals-overview#principal-types

Identity and Access Management
All APIs and reference
Authenticate to IAM
Retry failed requests
Client libraries
gcloud iam commands
REST API reference
RPC API reference
IAM Conditions reference
Roles and permissions reference
Full resource names
Identity federation: products and limitations
Permissions supported in deny policies
Permissions blocked by principal access boundary policies
Principal identifiers
Resource types that accept allow policies
Resource types with built-in identities
Service agents
Support levels for permissions in custom roles

Principal identifiers

Send feedback

When you refer to a principal in an Identity and Access Management (IAM) policy, you need to use the correct identifier for the principal. The format of the identifier depends on the type of principal that you want to refer to and the type of policy that you're writing.

This page lists the identifier formats for each policy type's supported principal types.

Principal identifiers for allow policies

The following table describes the principal identifiers for allow policies, which use the IAM `v1` API.

These identifiers are also used for Privileged Access Manager entitlements.

Preview — Principal identifiers for all service accounts in a project, folder, or organization

This feature is subject to the "Pre-GA Offerings Terms" in the General Service Terms section of the Service Specific Terms. Pre-GA features are available "as is" and might have limited support. For more information, see the launch stage descriptions.

Principal type	Identifier
Google Account	`user:USER_EMAIL_ADDRESS` Example: `user:alex@example.com`
Service account	`serviceAccount:SA_EMAIL_ADDRESS` Example: `serviceAccount:my-service-account@my-project.iam.gserviceaccount.com`

All service accounts in a project, folder, or organization	`principalSet://cloudresourcemanager.googleapis.com/RESOURCE_TYPE/RESOURCE_NUMBER/type/ServiceAccount`
	Example for all service accounts in a project:
	`principalSet://cloudresourcemanager.googleapis.com/projects/123456789012/type/ServiceAccount`
	Example for all service accounts in all projects in a folder:
	`principalSet://cloudresourcemanager.googleapis.com/folders/123456789012/type/ServiceAccount`
	Example for all service accounts in all projects in an organization:
	`principalSet://cloudresourcemanager.googleapis.com/organizations/123456789012/type/ServiceAccount`
	Note: Moving projects into or out of a folder or organization changes the service accounts included in this principal set. For example, if you move a project out of a folder or organization, then this principal set no longer includes that project's service accounts.

Identity pool and GKE workload Identity

a **Workload Identity Pool** is a Google Cloud resource that lets you manage identities from **outside of Google Cloud** (like AWS, Azure, GitHub Actions, or on-premises servers) so they can access GCP resources without needing risky, long-lived JSON service account keys.

A pool acts as a container for external identities. It consists of two main parts:

1. Pool Part 1: The Provider (The Source of Truth)

This is where you define who you trust. You configure the pool to listen to a specific external identity provider (IdP) using protocols like:

- **OIDC (OpenID Connect):** Used by GitHub Actions, GitLab, or Terraform Cloud.

- **SAML:** Used by enterprise identity providers like Okta or Active Directory.
- **AWS/Azure:** Direct integrations for workloads running on those clouds.

Note: in the case of GKE, Google simply "pre-builds" the pool (`PROJECT_ID.svc.id.goog`) (dedicated to GKE) and the provider (your Kubernetes cluster) so you don't have to set up the OIDC handshake manually.

User-Created Pools (External Identity Federation)

When you want to connect **AWS, Azure, or GitHub**, you create a **Workload Identity Pool** yourself.

- **Naming:** These do *not* use the `.svc.id.goog` suffix. They usually look like: `projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/MY-POOL-ID`
- **Purpose:** These are for "Generic" federation where *you* define the trust relationship and attribute mappings.

2. Pool Part 2: Attribute Mapping (The Translator)

Since an AWS Role or a GitHub Repo doesn't look like a Google identity, the pool "translates" their attributes into something Google understands.

- **Example:** You can map the GitHub `repository` attribute to a Google `attribute.repository`.

The Authentication Flow

When an external workload wants to talk to GCP, it follows this "Handshake" process:

1. **Identity Token:** The external workload (e.g., a GitHub Action) gets a signed token from its own provider.
2. **Exchange:** It sends that token to the **Workload Identity Pool**.

> In the context of GitHub Actions, the "configuration" that tells the CI/CD tool where to send its token is a mix of **Infrastructure setup** (on Google Cloud) and **Workflow definition** (on GitHub).

Neither GitHub nor Google Cloud "knows" about the other until you explicitly establish a trust relationship. Here is how both sides are rephrased to work together:

1. The Google Cloud Side: Building the "Front Door"

You create a **Workload Identity Provider** inside your pool. This acts as the specific "receptionist" for GitHub.

- **The Issuer URL:** You tell Google to only trust tokens issued by `https://token.actions.githubusercontent.com`. This is how Google knows the token is actually from GitHub.
- **The Provider ID:** Google generates a unique resource name for this provider (e.g., `projects/123/locations/global/workloadIdentityPools/my-pool/providers/my-provider`). **This is the destination address** you will give to GitHub later.

for example: github actions for cicd

1. Github actions 101:

To understand GitHub Actions, you only need to know these four terms:

- **Workflow:** The entire "playbook" or process, defined in a `.yml` file in your `.github/workflows` folder.
- **Events:** The "Trigger." This is what tells the robot to start (e.g., a `push`, a `pull_request`, or a scheduled time).
- **Jobs:** A workflow is made of one or more "Jobs." By default, these run in parallel, but you can make them wait for each other.
- **Steps/Actions:** These are the individual tasks inside a job. An **Action** is a reusable piece of code (like a plugin) that performs a complex task, such as "Login to Google Cloud".

Github runner: A **GitHub Runner** is the actual machine (the server) that executes the jobs defined in your GitHub Actions workflow.

2. The GitHub Side: Defining the "Handshake"

```
    GitHub Actions doesn't automatically reach out to Google. You must
configure your `.yml` workflow file with two specific components to
"drive" the token to the right place:
```

A. Requesting the Identity Token

```
    By default, a GitHub runner doesn't have a signed identity token.
You must explicitly grant it the permission to generate one in your
```

```
workflow YAML:

    YAML

    ```
    permissions:
      id-token: write # Required to request the JWT from GitHub's OIDC
provider
      contents: read   # Standard permission to read the repo
    ```
```

B. The "Authentication" Step

```
    You use a specialized action (like `google-github-actions/auth`) and
provide it with the **Provider ID** you created in Step 1.

    YAML

    ```
    - uses: 'google-github-actions/auth@v2'
      with:
        workload_identity_provider:
'projects/123/locations/global/workloadIdentityPools/my-
pool/providers/my-provider'
        service_account: 'my-sa@my-project.iam.gserviceaccount.com'
    ```
```

How it all connects at runtime

```
    1. **GitHub** sees the `id-token: write` permission and prepares a
signed token (JWT) containing your repo and branch info.

    2. The **Auth Action** takes that token and the
`workload_identity_provider` address you provided.

    3. The **Auth Action** "phones" Google's Security Token Service
(STS) at that specific provider address and hands over the GitHub token.

    4. **Google** checks its configuration: "Do I trust tokens from
`token.actions.githubusercontent.com`? Does this token come from the
```

```
specific GitHub repo I allowed?".

    5. If yes, **Google** hands back a short-lived Google Cloud token.
```

- **Validation:** The Pool checks if the token is valid and if it comes from a trusted provider.
- **STS (Security Token Service):** If valid, Google's STS exchanges that external token for a **short-lived Google access token**.
- **Access:** The workload uses that temporary token to call Google APIs (like uploading to a Bucket.

Why use a Pool instead of a Service Account Key?

Feature	Service Account Keys	Workload Identity Pools
Security	High risk; if stolen, they are valid until revoked.	Low risk; tokens are short-lived and expire automatically.
Management	You have to rotate and store secrets (Vault, etc.).	Keyless; no secrets to store or rotate.
Governance	Hard to track who is using which key.	Granular; you can permit access based on specific GitHub repos or AWS Roles.

The Dedicated GKE Workload pool

GKE Workload Identity is actually just a **specialized, pre-configured version** of this technology. Google simply "pre-builds" the pool (`PROJECT_ID.svc.id.goog`) and the provider (your Kubernetes cluster) so you don't have to set up the OIDC handshake manually.

Your GKE workloads use a pool that is "known" to IAM but doesn't require manual setup in the Federation console. When you want to grant a Pod permissions, you reference it using this principal format:
`principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/subject/ns/NAMESPACE/sa/KSA_NAME`

1. The Reserved Pool (`PROJECT_ID.svc.id.goog`)

When you enable Workload Identity on a GKE cluster, Google **automatically** creates this specific pool for you.

- **Purpose:** It is hardcoded for Kubernetes-native identities.
- **Naming:** It *must* follow that exact format. You cannot rename it, and you cannot create a pool with that name manually for your own external providers (like AWS or GitHub).
- **Management:** Google manages the OIDC provider (your cluster) within this pool.
-

The "Same Name, Different Lane" Comparison

Feature	GKE Workload Identity Pool	Standard Workload Identity Pool
Pool ID	Always `PROJECT_ID.svc.id.goog`	User-defined (e.g., `github-actions-pool`)
Provider	Automatically added (Your Cluster)	Manually added (AWS, OIDC, SAML)
Setup	One-click / Single CLI flag	Multi-step mapping & trust setup
Principal Format	`principal://iam.googleapis.com/.../subject/ns/NAMESPACE/sa/KSA`	`principal://iam.googleapis.com/.../subject/REPO_NAME`

--workload-pool

`--workload-pool` = `WORKLOAD_POOL`

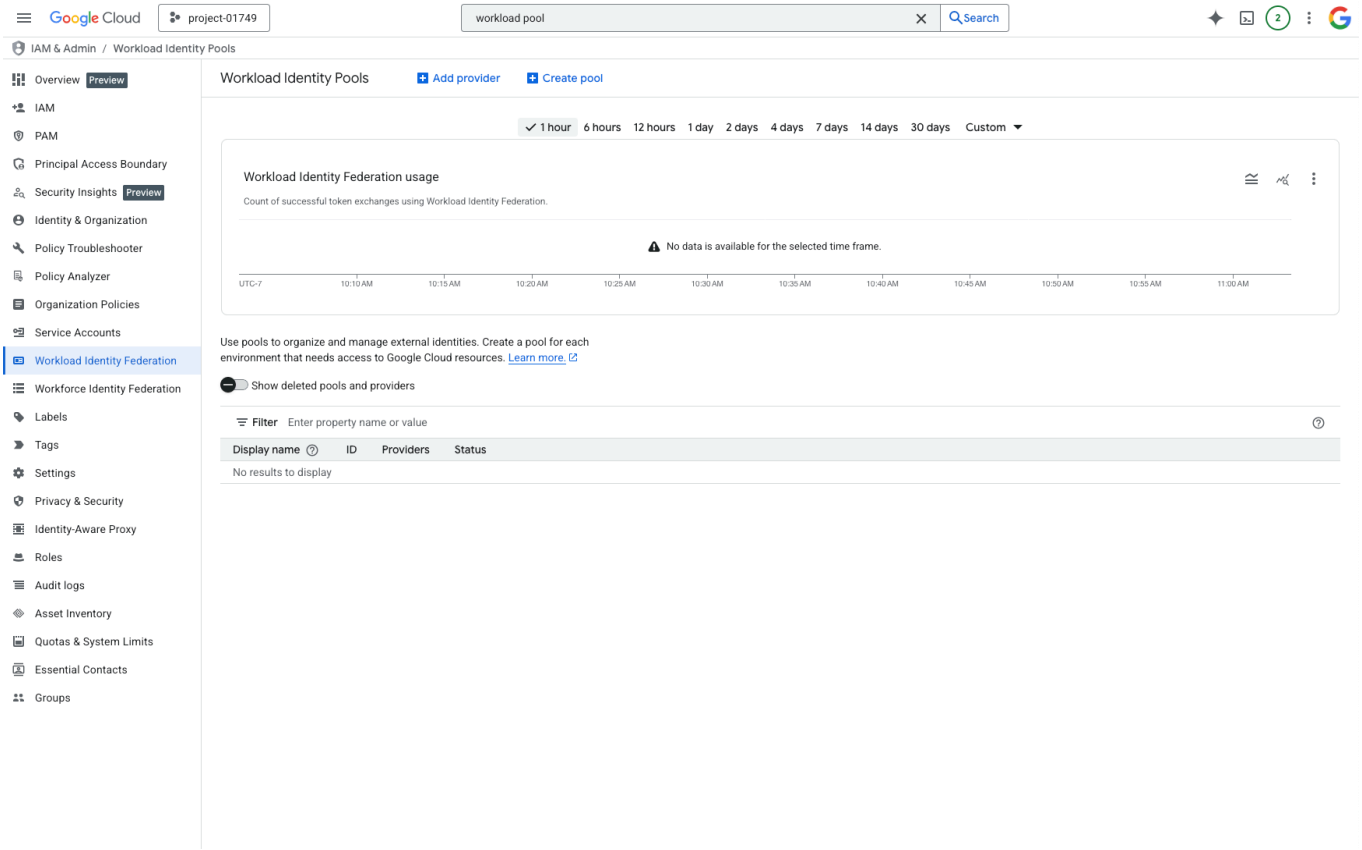
Enable Workload Identity on the cluster.

When enabled, Kubernetes service accounts will be able to act as Cloud IAM Service Accounts, through the provided workload pool.

Currently, the only accepted workload pool is the workload pool of the Cloud project containing the cluster, `PROJECT_ID.svc.id.goog`.

Workload Identity was enabled on your cluster because you included the `--workload-pool="project-01749.svc.id.goog"` flag.

Not in the console (hidden)



Why you don't see it in the "Workload Identity Federation" tab

It is completely normal that your GKE pool does not appear in the screenshot you provided. Here is the distinction:

- **Workload Identity Federation (The tab in your image):** This is used for **external** identities, such as workloads running on AWS, Azure, or GitHub Actions that need to access Google Cloud resources.
- **GKE Workload Identity:** This is a **Google-managed** implementation of that same federation technology. GKE automatically creates a "fixed" pool for your project with the format `PROJECT_ID.svc.id.goog`. Because Google manages this pool and its provider for you, it is generally hidden from the standard "Workload Identity Federation" UI to prevent accidental modification.

1. List all Workload Identity Pools

To list the pools you've created for external federation (the ones that would show up in your screenshot), use:

```
gcloud iam workload-identity-pools list --location="global"
```

Note: If you haven't created any external pools, this will return 0 items.

2. Verify GKE's Workload Identity status

Since the GKE pool is managed, you check its status by describing your cluster directly:

```
gcloud container clusters describe "acp-dev" \
  --region "us-central1" \
  --format="value(workloadIdentityConfig.workloadPool)"
```

If this returns `project-01749.svc.id.goog`, then Workload Identity is active and the pool exists in the backend.

```
admin_@cloudshell:~ (project-01749)$ gcloud container clusters describe "acp-dev" --region "us-cen
id.goog'
project-01749.svc.id.goog
admin_@cloudshell:~ (project-01749)$
```

Why only one pool per project?

GKE enforces a 1:1 relationship between a project and its workload pool to maintain security boundaries. By strictly using `PROJECT_ID.svc.id.goog`, Google ensures that:

- **Namespace isolation:** A "default" namespace in Project A is distinct from a "default" namespace in Project B.
- **Global Uniqueness:** No two organizations can accidentally claim the same identity pool.

When you enable Workload Identity, GKE configures a **Metadata Server** on every node. Here is the lifecycle of how that pool is used:

1. **The Pod Request:** Your application pod asks for a Google Cloud resource (like a Cloud Storage bucket).
2. **The Token Exchange:** The pod presents its **Kubernetes ServiceAccount (KSA)** token to the GKE Metadata Server.
3. **The Pool Verification:** The Metadata Server sends this to IAM. IAM looks at the pool ID (`PROJECT_ID.svc.id.goog`) to verify that this Kubernetes cluster is actually authorized to represent that project.
4. **The IAM Impersonation:** If the KSA is "bound" to a **Google ServiceAccount (GSA)**, IAM issues a short-lived Google OAuth token.

---

3. The "Identity" Mapping

Inside this workload pool, your Kubernetes identities are represented in IAM using a specific principal format. If you want to grant a Pod permissions, you don't grant them to the Pod itself;

you grant them to the identity within the pool:

```
principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/subject/ns/NAMESPACE/sa/KSA_NAME
```

Other KSA Principals

see New KSA Principals

The new KSA principals

All Pods that use a specific Kubernetes ServiceAccount
principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/subject/ns/NAMESPACE/sa/SERVICEACCOUNT

All workloads in a cluster (regardless of KNS or KSA)
principalSet://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/kubernetes.cluster/https://container.googleapis.com/v1/projects/PROJECT_ID/locations/LOCATION/clusters/CLUSTER_NAME

All workloads in a namespace (regardless of KSA or cluster):
principalSet://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT-ID.svc.id.goog/namespace/KNS

One specific Kubernetes ServiceAccount by its UID:
principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/kubernetes.serviceaccount.uid/SERVICEACCOUNT_UID

And more..

Google

Confidential + Proprietary

and more
https://cloud.google.com/iam/docs/principal-identifiers

Identity and Access Management
All APIs and reference
Authenticate to IAM
Retry failed requests
Client libraries
gcloud iam commands
REST API reference
RPC API reference
IAM Conditions reference
Roles and permissions reference
Full resource names
Identity federation: products and limitations
Permissions supported in deny policies
Permissions blocked by principal access boundary policies
Principal identifiers
Resource types that accept allow policies
Resource types with built-in identities
Service agents
Support levels for permissions in custom roles

Principal identifiers

Send feedback

When you refer to a principal in an Identity and Access Management (IAM) policy, you need to use the correct identifier for the principal. The format of the identifier depends on the type of principal that you want to refer to and the type of policy that you're writing.

This page lists the identifier formats for each policy type's supported principal types.

Principal identifiers for allow policies

The following table describes the principal identifiers for allow policies, which use the IAM `v1` API.

These identifiers are also used for Privileged Access Manager entitlements.

Preview — Principal identifiers for all service accounts in a project, folder, or organization

This feature is subject to the "Pre-GA Offerings Terms" in the General Service Terms section of the Service Specific Terms. Pre-GA features are available "as is" and might have limited support. For more information, see the launch stage descriptions.

Principal type	Identifier
Google Account	`user:`*`USER_EMAIL_ADDRESS`* Example: `user:alex@example.com`
Service account	`serviceAccount:`*`SA_EMAIL_ADDRESS`* Example: `serviceAccount:my-service-account@my-project.iam.gserviceaccount.com`

All service accounts in a project, folder, or organization	`principalSet://cloudresourcemanager.googleapis.com/`*`RESOURCE_TYPE`*`/`*`RESOURCE_NUMBER`*`/type/ServiceAccount`
	Example for all service accounts in a project:
	`principalSet://cloudresourcemanager.googleapis.com/projects/123456789012/type/ServiceAccount`
	Example for all service accounts in all projects in a folder:
	`principalSet://cloudresourcemanager.googleapis.com/folders/123456789012/type/ServiceAccount`
	Example for all service accounts in all projects in an organization:
	`principalSet://cloudresourcemanager.googleapis.com/organizations/123456789012/type/ServiceAccount`
	Note: Moving projects into or out of a folder or organization changes the service accounts included in this principal set. For example, if you move a project out of a folder or organization, then this principal set no longer includes that project's service accounts.

Example

Example

1. Create a Kubernetes **namespace** and a **Kubernetes Service Account (KSA)** (optional).

```
kubectl create namespace test_ns
kubectl create serviceaccount test_ksa --namespace test_ns
```

2. Assign the necessary **IAM roles** directly to the **KSA**

```
gcloud projects add-iam-policy-binding projects/test_project \
      --role=roles/appengine.appAdmin \
      --member=principal://iam.googleapis.com/projects/255719237156/locations
      /global/workloadIdentityPools/test_project.svc.id.goog/subject/ns/test_
      ns/sa/test_ksa \
      --condition=None
```

Google

Confidential + Proprietary

Namespace and KSA uniqueness across clusters

Question: if i have two clusters in the same project and each cluster has the same k8s namespace provisioned. Say test

i have workload in that name space with a KSA called admin.
That would be an issue for workload identity as the principals are not the same but the sytnax is the same?
`principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/subject/ns/NAMESPACE/sa/KSA_NAME`
NAMESPACE would be the same
KSA_NAME would be the same.
yet the intent is for cluster A workload in that namespace to have access to a Google 1P service such as cloud storage or bigquery while the workload in the other cluster would not.

Answer:

Actually, you've hit on one of the most important security nuances of GKE Workload Identity. Because the **Workload Identity Pool** is shared at the **Project** level (using the same `PROJECT_ID.svc.id.goog` name), the principal identifier for a Kubernetes Service Account (KSA) does not inherently include the cluster name.

As you noted, the principal for both clusters would be: `principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/subject/ns/test/sa/admin`

Is this a security issue?

Yes, if you use the default principal format. If you grant an IAM role to that specific principal, **both** clusters will have that permission because they both present the same identity to Google Cloud.

How to solve this: The "Cluster-Specific" Principal

To achieve your goal where Cluster A has access but Cluster B does not, you must use a more granular principal format that includes the **Cluster Name** and **Location**.

Instead of the standard `subject` (which only looks at namespace/KSA), you can use the `attribute.cluster_name` and `attribute.namespace` identifiers in your IAM binding.

1. The Granular Principal Syntax

You can bind the IAM role to a principal that specifies the cluster name as an attribute: `principalSet://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/attribute.cluster_name/CLUSTER_NAME`

2. Applying it to your scenario

To give **Cluster A** access to BigQuery, but deny **Cluster B**, you would run a command similar to this:

Default/commonly used Principal: `principal://iam.googleapis.com/projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/PROJECT_ID.svc.id.goog/subject/ns/test/sa/admin`

Grokking Workload Identity (old?)

https://docs.cloud.google.com/kubernetes-engine/docs/how-to/workload-identity#kubernetes-sa-to-iam
local copy:

https://docs.google.com/presentation/d/1l0ekKHcCsSffuLfMnqXFVKxR73iEosXTMWnQuvu9Mgo/edit?usp=sharing

Kubernete Tokens exchanged for GCP Token

Workload Identity

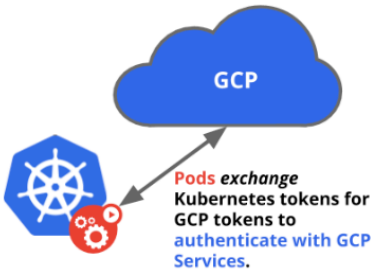
Identity Federation mechanism that allows k8s workloads to securely access GCP APIs

See also:

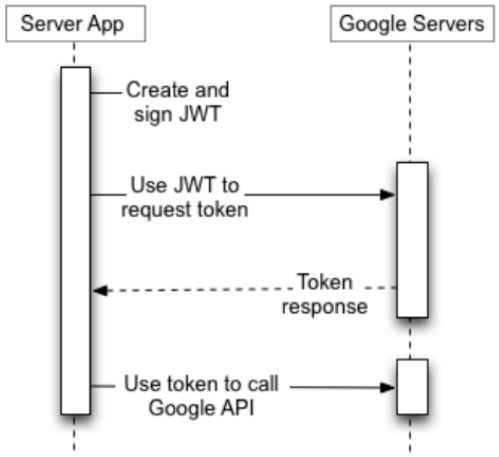
go/3pidentity

go/byoid-intro

go/anthos-identity-platform



GCP API authentication for server workloads



http://go/signinapi/protocols/oauth2/service-account

Access token request and response:

```
Access Token Request =>

POST /token HTTP/1.1
Host: oauth2.googleapis.com

grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer&
assertion=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiI3NjEzMjY3OTgwNjktcjVtbGpsbG4xcm
Q0bHJiaGc3NWVmZ2lncDM2bTc4ajVAZGV2ZWxvcGVyLmdzZXJ2aWNlYWNjb3VudC5jb20iLCJzY29wZSI6Imh0dHB
zOi8vd3d3Lmdvb2dsZWFwaXMuY29tL2F1dGgvcHJlZGljdGlvbiIsImF1ZCI6Imh0dHBzOi8vYWNjb3VudHMuZ29v
Z2xlLmNvbS9vL29hdXRoMi90b2tlbiIsImV4cCI6MTMyODU3MzM4MSwiaWF0IjoxMzI4NTY5NzgxfQ.ixOUGehweE
VX_UKXv5BbbwVEdcz6AYS-6uQV6fGorGKrHf3LIJnyREw9evE-gs2bmMaQI5_UbabvI4k-mQE4kBqtmSpTzxYBL1T
Cd7Kv5nTZoUC1CmwmWCFqT9RE6D7XSgPUh_jF1qskLa2w0rxMSjwruNKbysgRNctZPln7cqQ

Response =>

{
  "access_token": "1/8xbJqaOZXSUZbHLl5EOtu1pxz3fmmetKx9W8CV4t79M",
  "token_type": "Bearer",
  "expires_in": 3600
}

API call =>
curl -H "Authorization: Bearer $access_token" https://www.googleapis.com/drive/v2/files
```

Access without WI: Service Account Keys.

GCP API access from k8s *without* Workload Identity

---

https://cloud.google.com/kubernetes-engine/docs/tutorials/authenticating-to-cloud-platform

- Create a GCP Service Account (GSA)
- Create Keys for GSA
- Import GSA Keys as a k8s Secret
- For the k8s Workload:
  - Define a Volume with the Secret
  - Mount the Volume inside the container
  - Point $GOOGLE_APPLICATION_CREDENTIALS at the key file
- Workload can now authenticate to GCP APIs as the GSA

=> toilsome to setup & hard to secure

⚠ Warning

> long lived key, wide exposure: API server, any process within the Pod can read key

API access with WI 2.0: KSA and GSA binding

API access *with* Workload Identity

- Enable Workload Identity for the GKE cluster
- Run workload using a dedicated k8s service account (KSA)
- Grant KSA access to desired GCP resources using IAM roles
- Workload can now access GCP APIs by presenting (short-lived, auto-rotated) KSA tokens

It just works! *

*Not all GCP services can directly accept k8s tokens, yet. Needs workaround.

Enable Workload Identity for the GKE cluster
Run workload using a dedicated k8s service account (KSA)
Grant KSA access to desired GCP resources using IAM roles
Workload can now access GCP APIs by presenting (short-lived, auto-rotated) KSA tokens

---

Solution for APIs that don't support k8s Tokens

- Create a Google Service Account (GSA)
- Add KSA to the IAM role iam.workloadIdentityUser* allowing it to obtain tokens for the GSA
  - permission: 'iam.serviceAccounts.getAccessToken'
  - permission: 'iam.serviceAccounts.getCertificateAs'
  - permission: 'iam.serviceAccounts.getOpenIdToken'
- Transparently exchange KSA tokens for GSA tokens
- Workload can now access GCP APIs by presenting GSA tokens

```
* gcloud iam service-accounts add-iam-policy-binding \
  --role roles/iam.workloadIdentityUser \
  --member "serviceAccount:cluster_project.svc.id.goog[k8s_namespace/ksa_name]" \
  gsa_name@gsa_project.iam.gserviceaccount.com
```

GKE metadata server

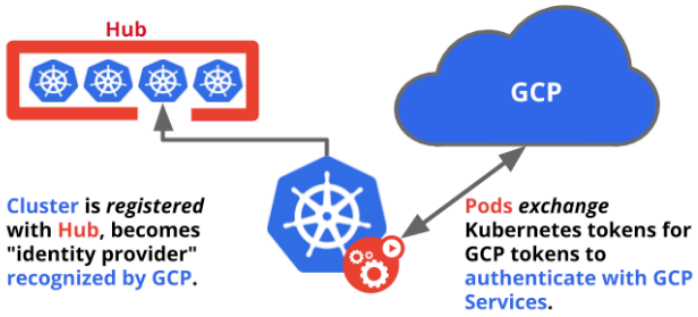
gke-metadata-server

- Runs as a DaemonSet
- iptables rule redirects Workload → GCE metadata server traffic to gke-metadata-server, which:
  - Looks up GSA using KSA annotation
  - Exchanges KSA token for Federated token RFC8693
  - Get GSA token by presenting Federated token
  - Return GSA token to Workload

```
kubectl annotate serviceaccount \
  --namespace k8s_namespace \
   ksa_name \
   iam.gke.io/gcp-service-account=gsa_name@gsa_project.iam.gserviceaccount.com
```

gke anywhere

go/gke-wi-anywhere-design



Workload Identity Anywhere go/gke-wi-anywhere-design

Any K8S cluster (on-prem, AWS, etc) can use Workload Identity using the general BYOID features

These clusters can then be joined to GKE Hub & take advantage of Anthos multi-cluster management features

Challenge: cluster's OIDC discovery endpoint needs to be accessible to GCP

GKE-on-prem solves this by caching these on a public GCS bucket